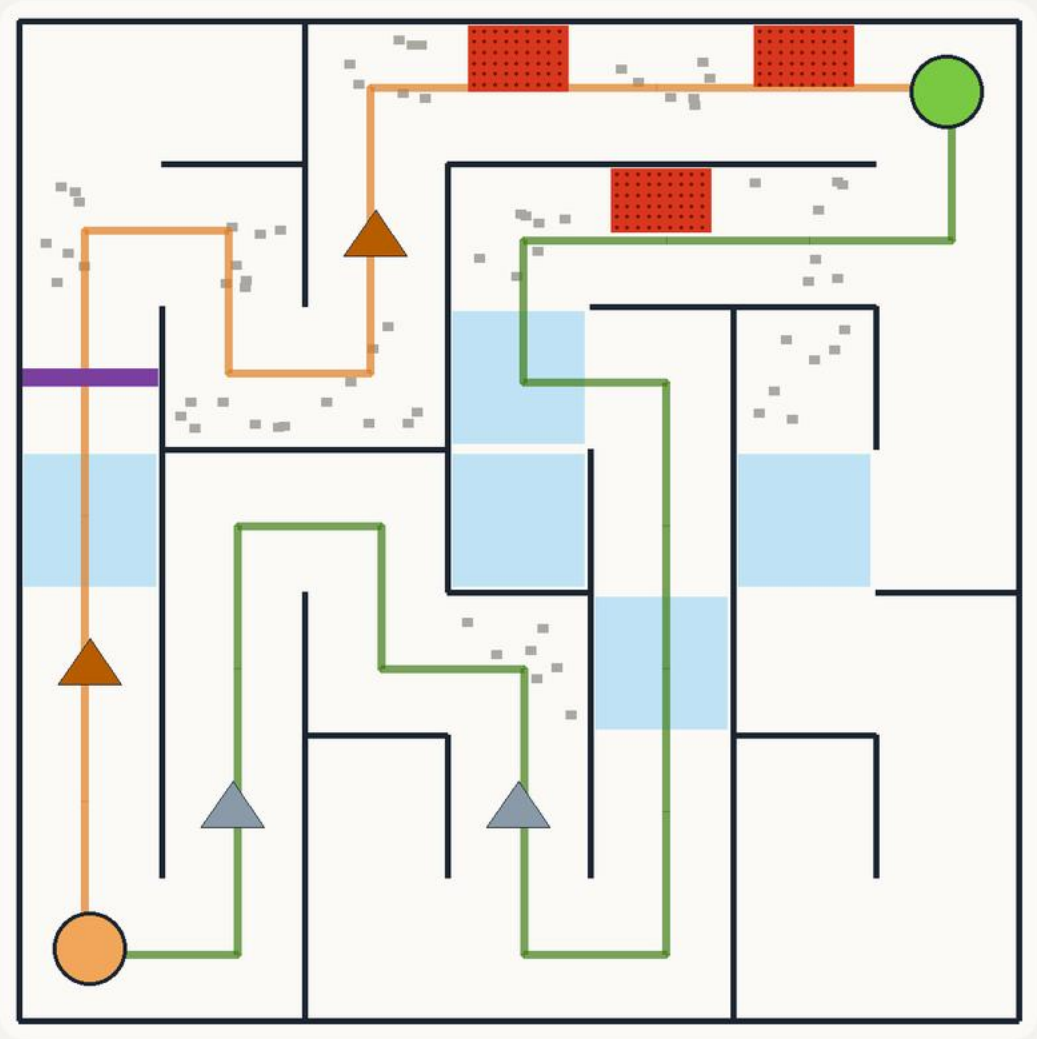
WEEK 7 · โจทย์พิเศษ CHALLENGE 7.5

G1 Hazard Maze

พาหุ่น G1 ออกจากเขาวงกตที่สุ่มใหม่ทุก seed มี 2 ทางให้เลือก ทางสั้นแต่ยาก กับทางยาวแต่ง่าย ส่ง policy กับ pilot มา แล้วแข่งจับเวลากันบนจอใหญ่



ด่าน 7 seed 1: เส้นส้ม = ทางสั้น เส้นเขียว = ทางยาว สามเหลี่ยม = เนิน (น้ำตาล = ชัน) แดง = แปลงหนาม ม่วง = ร่อง ฟ้า = น้ำแข็ง

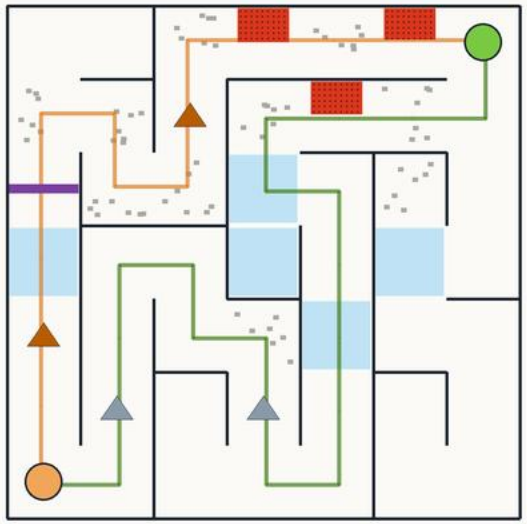
CHALLENGE RULES

ยากกว่า Lab 7.2 แทบทุกอย่าง

แผนที่	ซ้อม 7 × 7 และ 9 × 9 ด่านจริง 10 × 10 จุดเริ่ม/เส้นชัยสุ่ม (หน้าถัดไป) ออกจากจุดเริ่มได้ 2 ทาง ทางหนึ่งสั้นแต่ยาก อีกทางยาว เลี้ยวเยอะ ด่านสุ่มจาก seed ที่ครูประกาศ
สิ่งกีดขวาง	ทางสั้น: ขรุขระตลอด ร่อง เนินชัน 2 ลูก หนาม 2 แปลง (ชิ้นแรกใกล้ทางแยก) ทางยาว: เนินเตี้ย ของเกะกะ น้ำแข็ง **และหนามด้วย**
ชนะเมื่อ	ตัวหุ่นเข้าไปถึงกลางช่องเส้นชัย (ห่างไม่เกิน 0.6 m) ภายในเวลาของด่าน (300 / 480 / 600 s)
ล้ม / ชน / ค้าง	**แตะแปลงหนาม = ล้ม** ค้างคือขยับไม่ถึง 0.25 m ใน 10 วินาที โดนแล้วกลับไปเริ่มใหม่ **แต่เวลายังเดินต่อ**
จัดอันดับ	รันทีมละ 2 รอบ เอารอบที่ดีกว่า ใครถึงเร็วสุดชนะ ไปไม่ถึงดูว่าใครเหลือระยะถึงเส้นชัยน้อยกว่า (closest บนจอ) เสมอกันดูว่าใครล้มน้อยกว่า
ข้อมูลที่ใช้ได้	height scan, lidar และความเร็วของหุ่นเอง ห้ามแอบอ่านแผนที่จริง จำได้แค่ในรอบนั้น ห้ามเขียนไฟล์ไว้ใช้รอบหน้า

LEVELS

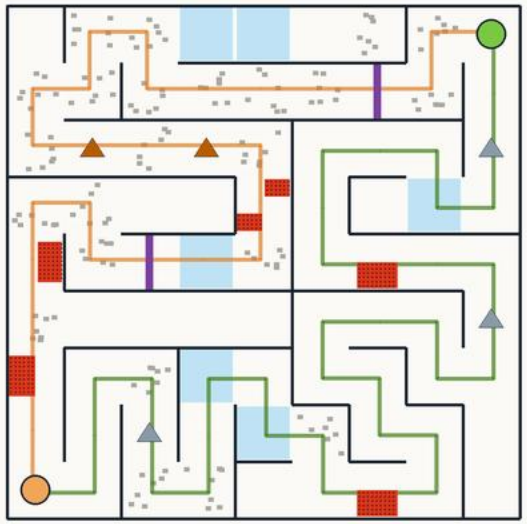
ซ้อม 2 ด่าน แล้วแข่งจริงบนด่านที่ใหญ่กว่า



--level 7 · ซ้อม

7 × 7 ช่อง · 14 × 14 m

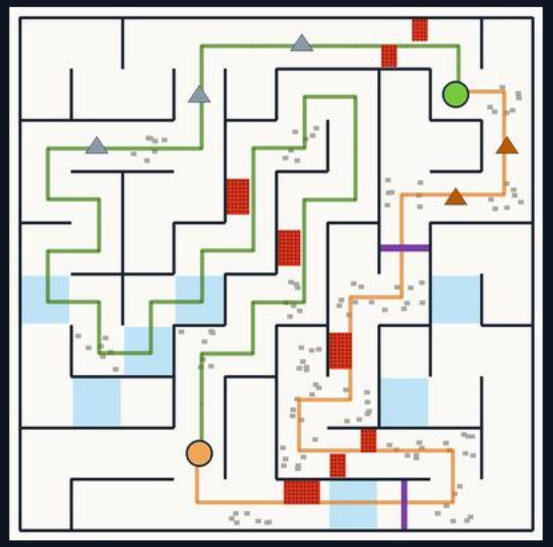
เริ่มมุมซ้ายล่าง เส้นชัยมุมขวาบน · 300 s
ทางยาวก็มีหนาม 1 แปลง



--level 9 · ซ้อม

9 × 9 ช่อง · 18 × 18 m

มุมเดิม · 480 s · ร่อง 2 จุด
หนามแบบสลาลม + หนาม 2 แปลงบนทางยาว



ด่านจริง · วันแข่ง

10 × 10 ช่อง · 20 × 20 m

จุดเริ่มและเส้นชัยสุ่ม · 600 s
ภาพนี้แค่ตัวอย่าง ด่านจริงประกาศวันแข่ง

ทุกด่าน: วงกลมส้ม = จุดเริ่ม เขียว = เส้นชัย เส้นส้ม = ทางสั้น (ยาก) เส้นเขียว = ทางยาว สี่เหลี่ยมแดง = แปลงหนาม (แตะ = ล้ม) ม่วง = ร่อง ฟ้า = น้ำแข็ง

FINAL LEVEL · ภาพประกอบ

ด่านจริง: ใหญ่ขึ้น จุดเริ่มกับเส้นชัยสุ่ม หนามทุกทาง



ตัวอย่าง 3 seed: จุดเริ่ม (วงกลมส้ม) กับเส้นชัย (เขียว) อยู่คนละที่ทุกครั้ง ห่างกันอย่างน้อย 10 ช่อง ทางสั้น 23–25 ช่อง ทางยาว 32–49 ช่อง

อะไร	ด่านจริง
ขนาด	10 × 10 ช่อง (20 × 20 m)
สุ่มจาก seed	กำแพง, จุดเริ่ม, เส้นชัย, ตำแหน่งและขนาดอุปสรรค
ทางสั้น	ร่อง 2 · เนินชัน 2 · หนาม 2 + สลาลม 1 · ขรุขระตลอด
ทางยาว	เนินเตี้ย 3 · หนาม 2 + สลาลม 1 · ของเกะกะ · น้ำแข็ง
เวลา	600 วินาที

pilot รู้อะไรตอนเริ่ม: ขนาดด่าน, ช่องเริ่ม, ช่องเส้นชัย

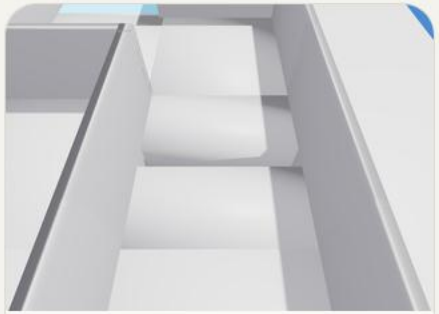
`Pilot(..., start_cell, goal_cell) · obs["goal_cell"]`

ที่เหลือ (กำแพง อุปสรรค ทางไหนง่าย) ต้องดูเอาจาก sensor ระหว่างเดิน

ไม่มีทางไหนปลอดหนาม: ต้องหลบให้เป็นทั้งแบบแปลงข้างเดียวและแบบสลาลม

HAZARDS

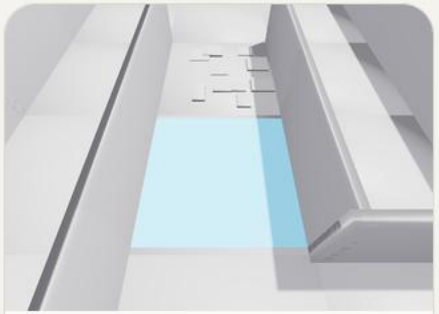
ทางสั้นเจออะไร ทางยาวเจออะไร



gap · ทางสั้น 15–22 cm

ร่องขวางทาง

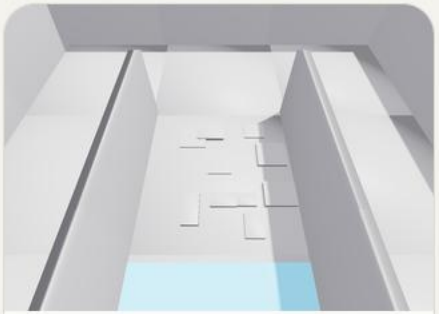
อยู่บนทางสั้น ลึก 1 m เท้าตกร่องเมื่อไหร่ล้มแน่ ต้องก้าวข้ามให้ได้



ice · สองทาง μ = 0.2

น้ำแข็ง

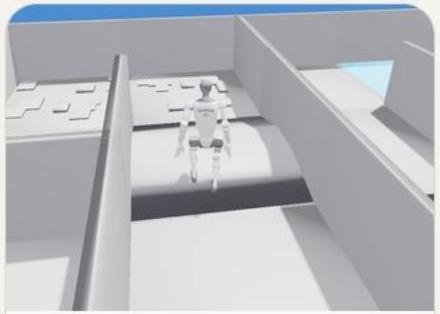
แผ่นสีฟ้าลื่นกว่าพื้นปกติ 5 เท่า เลี้ยวหรือเร่งบนนี้แล้วไถล



rough · 2–8 cm

ของเกะกะ

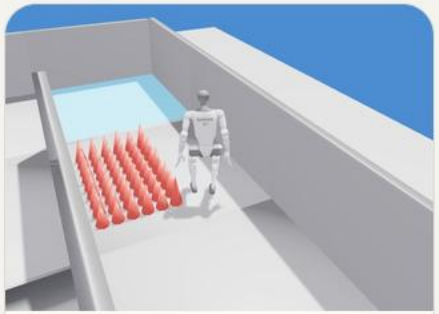
ทางสั้นขรุขระทุกช่องที่ว่าง ทางยาวมีบางช่อง กล่องเล็ก ๆ ช่องละ 10 ชิ้น



ramp · 10–30 cm

เนิน

ทางสั้นชัน 24–30 cm G1 Rough ปีนไม่ขึ้น ทางยาวเตี้ย 10–18 cm ปีนสบาย



spike · แตะ = ล้ม

แปลงหนาม

กรวย 25–35 cm ฝั่งเดียวของทาง ส่วนไหนของหุ่นเข้าไปในแปลง = ล้ม ต้องเดินเลนที่เหลือ (กว้าง 0.93–1.03 m) ด่าน 9 และด่านจริงมีแบบสลาลม: 2 แปลงสลับข้าง ต้องข้ามฝั่งตรงช่องว่าง 0.6 m

หนามไม่ใช่แค่พื้นยากแล้ว: pilot เริ่มต้นเดินชนหนามแล้วกลับจุดเริ่มซ้ำ ๆ ต้องเห็นหนามจาก scan_z แล้วหลบเอง

ทำอะไรต่อ

ไม่ต้องเทรนใหม่ตั้งแต่ศูนย์ แก้ได้ 2 ส่วน

policy.pt · "ขา"

Fine-tune policy การเดิน

ฝึกขาให้เดินบนร่อง น้ำแข็ง เนิน และกรวยได้โดยไม่ล้ม เทรนต่อจาก G1 Rough ได้เลย

ใช้เนื้อหา

Week 3 G1 Rough การ play และคำสั่งความเร็ว

Week 5 ปรับ reward เช่น ให้ยกเท้าสูงขึ้นตอนข้ามกรวย

Week 7 Lab 7.1 curriculum ให้พื้นค่อย ๆ ยากขึ้น

เทรนรอบละราว 30 นาที (1,500 it บน RTX 3060 Laptop) ได้ไฟล์ `model_XXXX.pt`

pilot.py · "สมอง"

เขียน pilot ใหม่

ให้หุ่นตัดสินใจเองว่าจะไปทางไหน เดินเร็วแค่ไหน เขียนเป็นโค้ด Python ไม่ต้องเทรน

ใช้เนื้อหา

Week 7 Lab 7.2 maze autopilot และ flood fill

height scan (scan_z) ความสูงของพื้นรอบตัวหุ่น

ความเร็วจริง เทียบกับที่สั่ง จะรู้ว่ากำลังลื่น

ลองรันได้ทันที รอบละ 1–2 นาที ได้ไฟล์ `pilot.py`

แนะนำ: เริ่มจาก pilot ก่อนเพราะลองได้เร็ว ถ้ายังล้มบนพื้นบ่อยค่อย fine-tune policy จะส่ง G1 Rough เดิมเป็น policy.pt ก็ได้

ขั้นตอนของนักเรียน

จากเครื่องตัวเอง → หน้าอัปโหลด → จอใหญ่

1

ลอง baseline

รันของเดิมดูก่อนว่าพังตรงไหน ถ้าไม่ใส่ --team จะเป็นแค่การทดสอบ ไม่ลงตาราง

```
uv run python
scripts/
challenge.py --
level 7 --seed 1
--viz newton
```

2

แก้ policy

fine-tune ต่อจาก G1 Rough จะได้ไฟล์ model_XXXX.pt ใน logs/rsl_rl/...

```
uv run isaaclab
train … --
checkpoint
pretrained/
g1_rough/
model_pretrained.pt
```

3

แก้ pilot (ถ้าอยากแก้)

ก๊อป DefaultPilot จาก scripts/challenge.py ไปเป็นไฟล์ pilot.py แล้วตั้งชื่อ class ว่า Pilot

4

ทดสอบหลาย seed

รันแบบไม่เปิดหน้าต่าง (เร็วขึ้นราว 2 เท่า) ดูบรรทัด RESULT ทุก seed ก่อนส่ง

```
uv run python
scripts/
challenge.py --
level 9 --policy
<model.pt> --pilot
pilot.py --seed 3
```

5

อัปโหลด

เปิดเบราว์เซอร์ไปที่ http://<IP ครู>:8000 ใส่ชื่อทีม แนบ .pt กับ pilot.py แล้วกด Upload

สรุป: ส่งแค่ **policy.pt** (ต้องมี) กับ **pilot.py** (ไม่บังคับ) ไม่ต้องส่งทั้งโปรเจกต์ ตัวด่านกับตัวรันใช้ของครู

FINE-TUNE POLICY

4 ขั้น: พื้นฝึก → task → เทรน → ทดสอบ

1

สร้างพื้นฝึก

เริ่มจาก tasks/hazard_starter/ (ร่อง เนิน กล่อง พื้นลื่น) แล้วแก้ให้เหมือนด่านมากขึ้น เช่น ใส่กรวย ทำเนินแบบในด่าน

2

ลงทะเบียน task

ก๊อปโฟลเดอร์ starter เป็นชื่อใหม่ แล้วเปลี่ยน id ใน __init__.py เป็น "MyHazard-Train" (starter เทรนได้ทันทีด้วย G1-Hazard-Starter)

3

เทรนต่อจาก G1 Rough

--checkpoint คือเริ่มจากขาที่เดินเป็นแล้ว 1,500 it กับหุ่น 1,024 ตัว ใช้ราว 30 นาที

4

ทดสอบ → วนกลับ

รัน challenge.py หลาย ๆ seed แล้วดู log fall/crash in cell (i, j) ว่าล้มตรงไหน แล้วเพิ่มอุปสรรคแบบนั้นในพื้นฝึก

```
class MyHazardEnvCfg(G1RoughEnvCfg): # ดูของจริงใน tasks/hazard_starter/env_cfg.py · obs / action ห้ามเปลี่ยน
    def __post_init__(self):
        super().__post_init__()
        self.scene.terrain.terrain_generator = MY_TERRAIN # ชั้น 1
        cmd = self.commands.base_velocity
        cmd.ranges.lin_vel_x = (0.0, 0.8) # ตรงกับที่ pilot สั่ง
        # heading_command เปิดไว้เหมือน G1 Rough (ปิดแล้ว terrain curriculum ฟัง)

$ uv run isaaclab train --rl_library rsl_rl --task MyHazard-Train --num_envs 1024 \
    --checkpoint pretrained/g1_rough/model_pretrained.pt agent.max_iterations=1500
```

PILOT.PY

ไฟล์เดียว class เดียว มี 3 เมธอด

```python
from g1_maze_lab.autopilot import MazeAutopilot

class Pilot:  # this name only
    def __init__(self, num_envs, num_cells, cell_size, device,
                 start_cell, goal_cell):
        self.ap = MazeAutopilot(num_envs, num_cells, cell_size,
            device, strategy="flood",
            start=start_cell, goal=goal_cell)

    def act(self, obs):  # every 0.02 s
        p, ap = obs["pos_xy"], self.ap
        walls = ap.walls_from_height_scan(
            obs["scan_xy"], obs["scan_z"], ap.cell_of(p))
        return ap.step(p, obs["yaw"], walls, obs["dt"])

    def reset(self, env_ids):  # after a fall
        self.ap.reset(env_ids)
```

obs[...]	คืออะไร
pos_xy, yaw	ตำแหน่ง (m) จากกลางช่อง (0, 0) และทิศที่หัน
start_cell, goal_cell	ช่องเริ่ม / เส้นชัย (i, j)
scan_xy, scan_z	height scan 33 × 33 จุด
lidar_dist, lidar_yaw	lidar 72 เส้น ระดับอก
lin_vel_b, ang_vel_b	ความเร็วจริงของลำตัว
t, dt	เวลาที่ผ่านไป, 0.02 s ต่อรอบ

return

คำสั่งความเร็ว (N, 3)

v_x, v_y (m/s) และ w_z (rad/s) เทียบกับตัวหุ่น แล้ว policy จะแปลงเป็นท่าเดินเอง

หมายเหตุ: โค้ดนี้คือ pilot เริ่มต้นแบบย่อ ตัวเต็มอยู่ใน DefaultPilot ของ scripts/challenge.py แนบไฟล์ .py ที่ pilot import เพิ่มได้อีก 5 ไฟล์

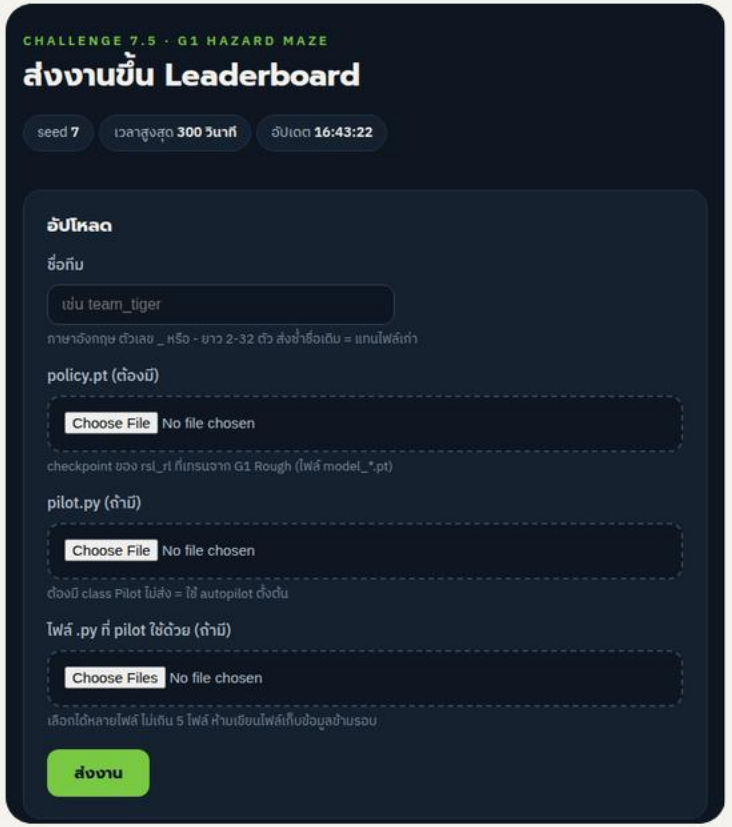
UPLOAD

ส่งงานแล้วดูผลบนหน้าเดียวกัน

1. เปิด http://<IP เครื่องครู>:8000 ต่อ Wi-Fi วงเดียวกับเครื่องครู
2. **ชื่อทีม** ภาษาอังกฤษ ตัวเลข _ - เช่น team_tiger ส่งทุกครั้งใช้ชื่อเดิม
3. **policy.pt** เลือก model_XXXX.pt ได้เลย ระบบเช็คให้ว่า network ตรงกับ G1 Rough
4. **pilot.py** ถ้ามีก็แนบ ไฟล์ .py ที่ pilot เรียกใช้ แนบเพิ่มได้อีก 5 ไฟล์
5. กด **ส่งงาน** กรอบเขียวแปลว่าส่งแล้ว กรอบแดงจะบอกว่าผิดตรงไหน แก้แล้วส่งใหม่

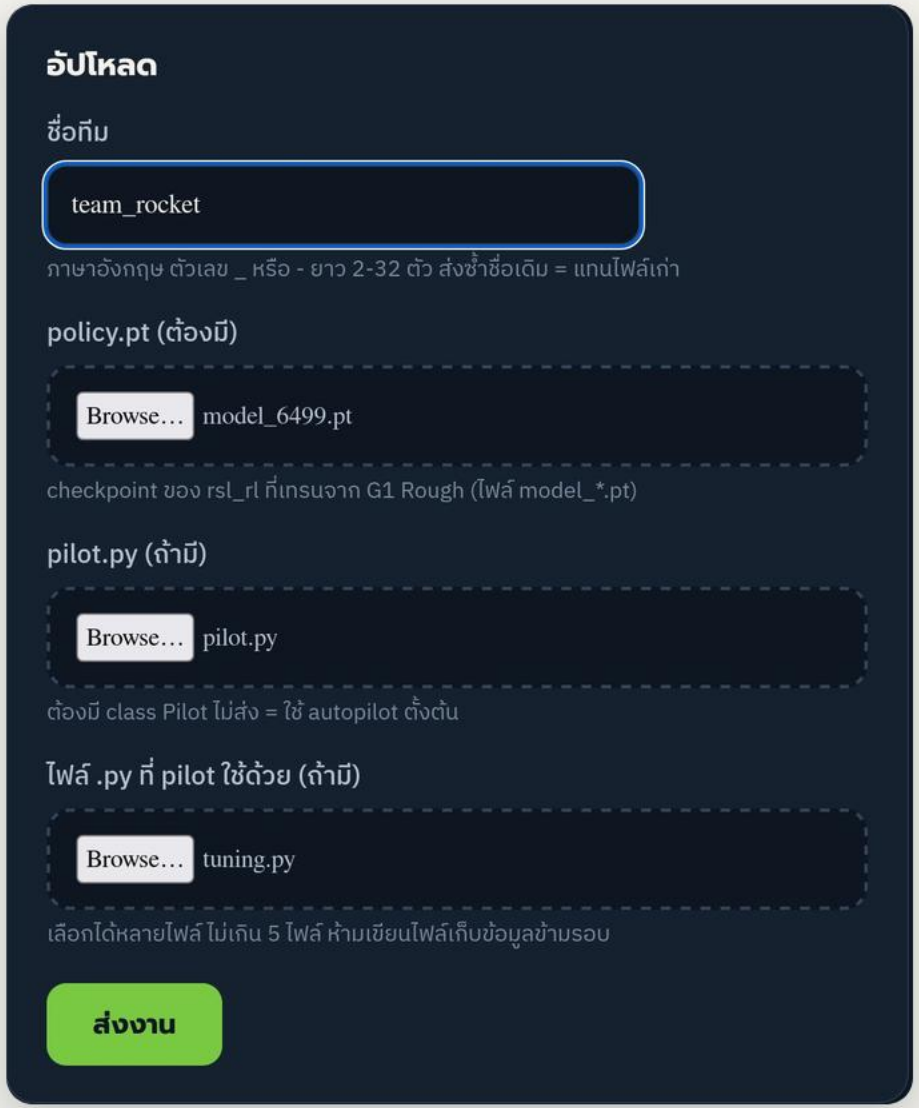
ดูสถานะทีมเราในตาราง

รอรัน แล้ว **กำลังรัน** แล้ว **รันแล้ว** พร้อมเวลา ถ้าขึ้น **error** จะบอกสาเหตุ เช่น pilot พังตอนวิ่ง ตารางนับเฉพาะ seed ของรอบนี้ ใช้รอบที่ดีที่สุดของแต่ละทีม ส่งใหม่ได้เรื่อย ๆ ไฟล์ใหม่ทับไฟล์เก่า



ครูเปิด /board ขึ้นโปรเจกเตอร์ เป็นแท่นรางวัลกับตารางที่อัปเดตเองทุก 5 วินาที

ก่อนส่ง: ห้ามเปลี่ยน input / output ของ policy (ต้องเหมือน G1 Rough) และลองรันหลาย seed ในเครื่องตัวเองก่อนทุกครั้ง



ส่งไฟล์อะไรบ้าง

ไฟล์เดียวที่ต้องมี กับอีก 2 ช่องที่ไม่บังคับ

policy.pt · ต้องมี

ไฟล์ model_XXXX.pt ตัวเลขมากสุดของรอบที่เทรน ชื่ออะไรก็ได้ ระบบเปลี่ยนเป็น policy.pt ให้

`logs/rsl_rl/g1_hazard_starter/<วันที่_ชื่อรอบ>/model_6499.pt`

ไม่ได้เทรน: pretrained/g1_rough/model_pretrained.pt

pilot.py · ถ้ามี

ต้องมี class Pilot ไม่ส่ง = ใช้ autopilot ตั้งต้น (DefaultPilot)

ไฟล์ .py ที่ pilot ใช้ · ถ้ามี

ไม่เกิน 5 ไฟล์ เช่น tuning.py ที่ pilot เรียกด้วย `from tuning import SPEED` ห้ามชื่อ pilot.py

ไม่ต้องส่ง: env_cfg.py, โฟลเดอร์ logs ทั้งก้อน, .venv ไฟล์พื้นฝึกใช้แค่ตอนเทรน ตอนแข่งใช้แค่ policy.pt

กดส่งแล้วเกิดอะไร

ส่งแล้ว → กำลังรัน → รันแล้ว ดูได้จากหน้าเดียวกัน

1. กรอบเขียว บอกไฟล์ที่ระบบรับ (กรอบแดง = ยังไม่รับ บอกเหตุผล)

✓ ส่งงานทีม **team_rocket** แล้ว: policy.pt, pilot.py, tuning.py
รอครูรัน ผลจะขึ้นในตารางด้านล่าง

2. แถบกำลังรัน ท้ายหน้า ทีมไหนอยู่บนเครื่องครูตอนนี้

● **กำลังรัน:** team_rocket

3. ตาราง เวลา ล้มกี่ครั้ง สถานะ รันแล้ว (error = บอกบรรทัดที่พัง)

#	ทีม	ผล	ล้ม	รอบที่รัน	สถานะ
1	team_rocket	77.7 s	0	1	รันแล้ว



ครูเปิด /board ขึ้นโปรเจกเตอร์ อัปเดตเองทุก 5 วินาที

รอบซ้อม: ส่งแล้วรันให้เองภายในไม่กี่วินาที ทีมละ 1–3 นาที ตามคิว ส่งใหม่ระหว่างทีมเรากำลังวิ่งไม่ได้

PRACTICE ROUND

รอบซ้อม: ส่งแล้วเครื่องครูรันให้เลย

```
# รอบซ้อม ด่าน 9 seed 1 · รันทุกงานที่ส่งเข้ามาทันที
$ CHALLENGE_LEVEL=9 CHALLENGE_SEED=1 uv run python scripts/
challenge_server.py
$ uv run python scripts/challenge_runner.py --level 9 --seed 1 \
    --watch --no-replay

# แข่งจริง: ปิดทั้งสองตัว แล้วเปิดใหม่ด้วย seed ที่ประกาศ
$ CHALLENGE_LEVEL=final CHALLENGE_SEED=<seed> uv run python scripts/
challenge_server.py
$ uv run python scripts/challenge_runner.py --level final --seed <seed>
```

ระบบไม่รับ ถ้า...	บนหน้าเว็บขึ้นว่า
policy ไม่ใช่ network ของ G1 Rough	ชั้นแรกรับกี่ค่า ต้องรับกี่ค่า
pilot.py syntax ผิด หรือไม่มี class Pilot	ไฟล์ไหน บรรทัดไหน
pilot ใช้แผนที่จริง (GENERATED, HAZARDS, g1_maze_lab.terrains)	บรรทัดที่ใช้
ชื่อทีมไม่ใช่ a-z 0-9 _ - หรือไฟล์ประกอบเกิน 5	สิ่งที่ต้องแก้

รอนานแค่ไหน

รันทีละทีมตามคิว ทีมที่ถึงเส้นชัยใช้ราว 1–2 นาที ทีมที่ไปไม่ถึงวิ่งครบ 300 วินาที ราว 3 นาที ดูคิวของทีมเราได้บนหน้าเว็บ ส่งใหม่ระหว่างที่ทีมเรากำลังวิ่งไม่ได้

รับแล้วแต่พังตอนวิ่ง

pilot error ระหว่างวิ่ง ตารางขึ้น **error** พร้อมบรรทัด error ค้างนานเกิน 18 นาที ระบบหยุดให้แล้วขึ้น error เหมือนกัน แก้แล้วส่งใหม่ได้

ใช้รอบซ้อมให้คุ้ม: ลองส่งไฟล์จริงสักรอบก่อนแข่ง จะได้รู้ว่าไฟล์ผ่าน และ pilot ไม่พังบนเครื่องครู

PLAY

ส่งครบแล้ว ครูรันทุกทีมตามคิว

```
# ครู เทอร์มินัล 1: หน้าส่งงาน + leaderboard
$ CHALLENGE_LEVEL=final CHALLENGE_SEED=7 uv run python \
    scripts/challenge_server.py
# เทอร์มินัล 2: ส่งครบแล้ว รันทุกทีมตามคิว อัดคลิปเก็บไว้ด้วย
$ uv run python scripts/challenge_runner.py \
    --level final --seed 7
[RUNNER] seed 7: tiger, oops, rocket, panda, ...
[RUNNER] ramp_squad: GOAL 71.5 s, falls 0
# ไม่เอาคลิป รันเร็วขึ้น: --no-replay
```

HUD

เวลา ระยะที่เหลือ และจำนวนครั้งที่ล้ม

to goal คือระยะที่เหลือตามทางที่กำลังเดิน closest คือใกล้ที่สุดที่เคยไปถึง เหลือไม่ถึง 30 วินาทีตัวเลขจะเป็นสีแดง ล้มแล้วขึ้น “FELL / CRASH” ถึงเส้นชัยแล้วนาฬิกาหยุด กด C เพื่อหมุนกล้องเอง



หน้าจอตอนรัน ด่าน seed 7 วินาทีที่ 60 หุ่นกลับมาใช้ทางยาว เลย เหลือ 43.4 m แต่เคยไปใกล้สุดที่ 16.1 m

REPLAY

รันเสร็จแล้ว เปิดรีเพลย์ดูพร้อมกันทั้งห้อง

1. ส่งงานแล้วหน้าเว็บบอกว่าทีมเราอยู่ **คิวที่เท่าไหร่** มาก่อนรันก่อน ส่งใหม่ไปต่อท้ายคิว
2. ครูรันทุกทีมทีเดียว หน้า /board บอกว่ากำลังรันทีมไหน ระหว่างนั้นอัดคลิปของทุกทีมเก็บไว้
3. เปิด /replay ขึ้นโปรเจกเตอร์ เล่นจากอันดับท้ายขึ้นไปหาแชมป์ ผลเฉลยตอนจบคลิป ปรับความเร็วได้ 1x ถึง 8x
4. กดปุ่ม ▶ ในตาราง ดูคลิปของทีมไหนก็ได้ จากมือถือก็ได้

เผื่อเวลา: อัดคลิปไปด้วย แต่ละทีมใช้เวลาราว 1.2 เท่าของเวลาในด่าน ทีมที่ไปไม่ถึงวิ่งครบ 300 วินาที ราว 6 นาทีต่อทีม



คลิปตัวอย่าง: ส่งงาน รันตามคิว แล้วเปิดรีเพลย์ ผลจากรอบทดสอบจริง seed 7 ทีมจำลอง 7 ทีม

EXAMPLE RUN · V4

หนามตายแล้ว pilot ตั้งต้นไปไม่ถึงสักด่าน

ด่าน · seed	pilot ตั้งต้น (DefaultPilot)	เฉลยของครู (หลบหนาม + จำช่องที่ล้ม)
7 · seed 7	ไม่ถึง (เหลือ 20.9 m) ล้ม 51 ครั้ง เป็นหนาม 49	**157.0 s** ล้ม 2 ไม่โดนหนาม
9 · seed 7	ไม่ถึง (เหลือ 60.9 m) ล้ม 69 เป็นหนาม 66	ไม่ถึง (เหลือ 39.9 m) ล้ม 4 ไม่โดนหนาม
ด่านจริงตัวอย่าง · seed 7	ไม่ถึง (เหลือ 44.9 m) ล้ม 17	ไม่ถึง (เหลือ 46.0 m) ล้ม 4 โดนหนาม 1
ด่านจริงตัวอย่าง · seed 21	–	ไม่ถึง (เหลือ 4.0 m) ล้ม 3 โดนหนาม 1

pilot ตั้งต้นพังตรงไหน

เดินกลางทางตรงเข้าแปลงหนามช่องแรกของทางสั้น กลับจุดเริ่มแล้วก็เดินเข้าไปใหม่ทุก 6 วินาที เพราะไม่จำว่าเคยตายตรงไหน

ด่านใหญ่ยากที่ "หาทาง"

เฉลยหลบหนามได้แทบทุกครั้ง แต่ 9 × 9 และ 10 × 10 มีทางตันเยอะ flood fill เดินสำรวจจนหมดเวลา ทีมที่วางแผนเส้นทางเก่งกว่านี้มีโอกาสชนะ

โจทย์ยังเปิด: ด่าน 9 และด่านจริง ยังไม่มีใครพาหุ่นถึงเส้นชัย รวมทั้งเฉลยของครูเอง

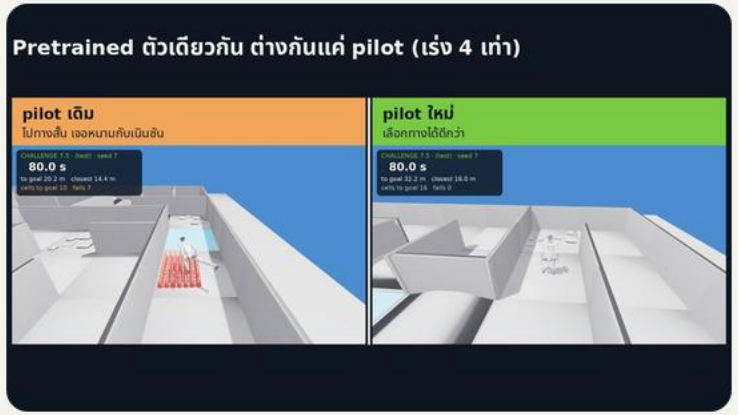
VIDEO

เลือกทางให้ดี หรือฝึกให้ปีนเนินไหว



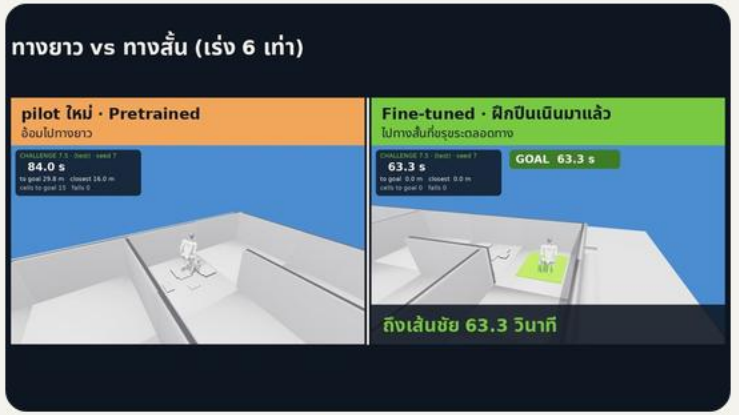
ใครจะพาหุ่นออกไปได้เร็วสุด?

ไฮไลต์ 8 วินาที: หนามลื่นเหยียบแล้วล้ม เนินชัน 26 cm แล้วก็ถึงเส้นชัย



pilot เดิม vs pilot ใหม่

หุ่น Pretrained ตัวเดียวกัน ซ้ายเหยียบหนามล้มซ้ำ ๆ ขวาเลือกทางได้ดีกว่า ถึงใน **136.9 วินาที** เร่ง 4 เท่า



ทางยาว vs ทางสั้น

Fine-tuned ที่ฝึกปีนเนินชันมาแล้ว ไปทางสั้นถึงใน **63.3 วินาที** เร็วกว่าทางยาวเกินเท่าตัว เร่ง 6 เท่า

จัดอันดับ: ใครถึงเร็วสุดชนะ ไปไม่ถึงดูว่าใครเหลือระยะถึงเส้นชัยน้อยกว่า เสมอกันดูว่าใครล้มน้อยกว่า

ระยะที่เหลือนับตามทางที่หุ่นกำลังเดิน ไปทางยาวก็ไม่เสียเปรียบ ล้มกลับไปเริ่มใหม่ ค่า closest ก็ไม่หาย

HINTS

เริ่มจากตรงไหนดี

1 · ดูก่อนแก้

หุ่นพังตรงไหน

รันแบบ --viz newton แล้วดูว่าล้มที่ร่อง น้ำแข็ง เนิน หนาม หรือชนกำแพง ใน log จะบอกช่องที่ล้ม fall/crash #n in cell (i, j)

2 · policy

พื้นที่ฝึก = พื้นที่เจอ

อะไรที่ไม่เคยเจอตอนเทรนก็มักเดินไม่ผ่าน ดูวิธีในหน้า Fine-tune 4 ขั้น

3 · pilot

ช้าลงตอนที่ควรช้า

ใน obs มีความเร็วจริงของหุ่นเทียบกับที่สั่งไปจะรู้ว่ากำลังลื่นหรือติดอยู่ เดินเร็วสุดไม่ได้แปลว่าดีสุด

4 · pilot

ดูยังไงว่าทางไหนง่าย

ใน scan_z ทางง่ายจะไม่ค่อยมีอุปสรรค ลองเดินเข้าไปนิดเดียวแล้วดูพื้นข้างหน้าจาก height scan ถ้าเจอเนินชันหรือหนามตั้งแต่ต้นทาง แนะนำให้กลับไปเปลี่ยนเส้นทาง

อย่าลืม: ห้ามแก้ terrains/hazard_maze.py กติกาการล้ม เวลา หรือขนาดเขาวงกต และห้ามให้ pilot แอบอ่าน GENERATED/HAZARDS (แผนที่จริง) หรือเขียนไฟล์ไว้ใช้รอบหน้า

เบื้องหลังเฉลย · การเทรน

ปัญหาที่ครูเจอตอนเทรน แล้วแก้ยังไง

เจออะไร	ทำไมถึงเป็นแบบนั้น	แก้ยังไง
เรียนรู้ผิด: ฝึกปีนเนินแล้ว หุ่นไม่ยอมเข้าใกล้ร่อง สั่งเดินหน้าก็ยืนนิ่งห่างขอบ 0.6 m	ตอนฝึกมีพื้นร่องแค่ 25% และตกร่องโดนหักคะแนนหนัก หุ่นเลยเรียนว่า "อย่าเข้าใกล้ร่อง" ง่ายกว่าเรียนวิธีก้าวข้าม	เทรนต่อโดยเพิ่มพื้นร่องเป็น 45% ข้ามร่อง 15–20 cm ได้แล้ว แต่ 26 cm ยังไม่กล้า ครูเลยปรับร่องบนทางสั้นให้แคบลงเป็น 15–22 cm
ฝึกแล้วระดับความยากลดลงเรื่อย ๆ จาก 2.5 เหลือ 0.46	ปิด heading command ไว้ หุ่นได้คำสั่งให้หมุนแบบสุ่มเลยเดินเป็นเส้นโค้ง แต่ curriculum วัดระยะเป็นเส้นตรง เลยคิดว่าเดินได้ไม่ไกลแล้วลดระดับ	เปิด heading command กลับเหมือน G1 Rough ระดับขึ้นไปถึง 4.5 และปีนเนินชัน 26–27 cm ได้
เทรนไปครึ่งทางแล้วพัง ค่าที่หุ่นมองเห็นกลายเป็น NaN	ฟิสิกส์ระเบิดชั่วขณะในหุ่นบางตัวบนเนินชัน	เทรนต่อจาก checkpoint ล่าสุด (บันทึกทุก 50 iteration) ไม่ต้องเริ่มใหม่
G1 Rough ปีนเนินสูงเกิน 0.22 m ไม่ขึ้น	ตอนเทรนมาไม่เคยเจอเนินรูปทรงแบบในด่าน	ทดสอบหลายความสูง ความสูงละ 3 รอบ หาเพดานก่อน แล้วสร้างพื้นฝึกเป็นเนินรูปทรงเดียวกับในด่าน

บทเรียน: ฝึกของใหม่อาจทำให้ลืมของเก่า เทรนเสร็จทุกครั้งต้องลองกับทุกอุปสรรคในด่าน ไม่ใช่แค่อันที่เพิ่งฝึก

เบื้องหลังเฉลย · PILOT และด่าน

ปัญหาที่ไม่ได้แก้ด้วยการเทรน

เจออะไร	ทำไมถึงเป็นแบบนั้น	แก้ยังไง
หุ่นเลี้ยวช้า ตัวค้างตอนเลี้ยว	pilot เดินไปให้ถึงกลางช่องก่อน แล้วค่อยหมุนตัวอยู่กับที่ช้า ๆ	ให้ pilot เริ่มเลี้ยวก่อนถึงกลางช่อง และเดินต่อระหว่างเลี้ยว เลี้ยวเป็นโค้ง เร็วขึ้นราว 20%
หนามไม่ได้ทำให้ยากเลย หุ่นเดินลุยผ่านสบาย	กรวยเตี้ยแค่ 15–25 cm และไม่ลื่น เลยเป็นแค่พื้นขรุขระที่ G1 Rough ถนัด	ทำกรวยสูง 25–35 cm และลื่น แล้วลองให้หุ่นเดินบนหนามตรง ๆ ว่าล้มจริง
หุ่นติดอยู่ระหว่างเนินสองลูก	เนินอยู่ติดกันเลยกลายเป็นหลุมรูปตัว V ปีนออกไม่ได้	ตอนสร้างด่าน ห้ามวางเนิน หนาม และร่องติดกัน
หลบหนามไม่ทันหลังเลี้ยว	scan มองได้แค่ราว 1.6 m พอเลี้ยวเข้าช่องก็ใกล้หนามเกินไปแล้ว	ให้หุ่นจำสิ่งที่เห็นจากด้านข้างตั้งแต่ก่อนเลี้ยว แล้วค่อยไปตั้งหลักก่อนถึงหนาม

บทเรียน: ล้มทุกครั้งดู log ว่าล้มที่ช่องไหน แล้วเปิดดูวิดีโอตรงนั้น ส่วนใหญ่สาเหตุไม่ใช่สิ่งที่คิดไว้ตอนแรก